MILESTONE 3

What is RAG?

RAG, or Retrieval-Augmented Generation, is a cutting-edge artificial intelligence technique that enhances the capabilities of large language models (LLMs) by connecting them to external knowledge sources. In essence, it allows these models to "look up" information before answering a question, leading to more accurate, up-to-date, and contextually relevant responses.

Embedding models:

1. **Sentence-BERT (SBERT)**

   Main goal - Map semantically similar sentences or documents to *nearby points* in an embedding space. For example, *"formal blue shirt"* and *"navy office shirt"* → high cosine similarity.

   Strengths -

   - Works directly on **JSON field–value text** because each key-value pair is semantically meaningful.(but cannot directly use json example if we have customer_name-harsha, we need to transform it to customer harsha has some query)
   - Fast and easy to run locally.

   Limitations -

   - Limited context window (512 tokens)(only 300-400 words in a single embedding).

- Doesn't natively distinguish between "query" and "document" roles (both encoded the same way).

2. **E5 Family (Embeddings from Instructions):**

   E5 (from Microsoft & Intfloat) is based on **instruction-tuned contrastive learning**.

   Instead of only learning from sentence pairs, it learns from **instruction–response pairs**

   Main goal - Teach the model to produce embeddings that excel in information retrieval tasks, not just general similarity. It understands *search intent* better than SBERT — so "cheap formal shirt" retrieves "affordable officewear".

   Trained with - massive datasets from search engines, QA pairs, and web text, with explicit "query/passsage" formatting.

   Strengths -

   - Optimized for **retrieval and ranking**.
   - Outperforms SBERT on open-domain search tasks.
   - Great for **product search**, **recommendation**, and **semantic filtering**.

   Limitations -

   - Slightly heavier (~768 dimensions).
   - Requires you to prefix text ("query:" / "passage:") for optimal results.
   - Does not directly support json we need to modify it

3. **BGE Family (BAAI General Embedding)**

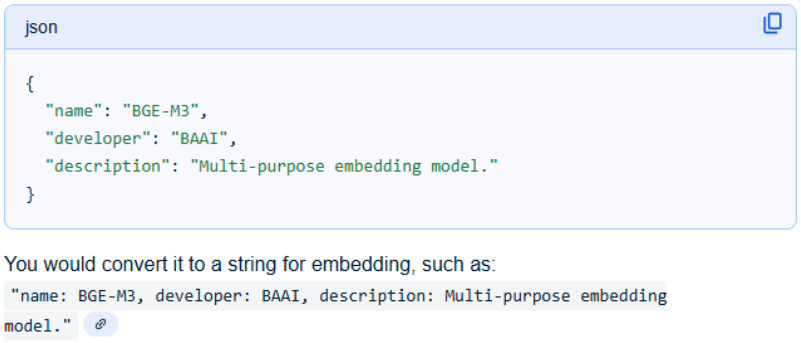
   Similar to E5 but tuned for **dense retrieval** efficiency.

   Main goal - Produce **retrieval-oriented embeddings** that maintain semantic fidelity even with small models

Trained with Large multi-domain dataset including natural text, Q/A pairs, web docs, and multilingual sources.

Strengths -

- Perfect for **vector search in ecommerce** (similar products, recommendations).
- Excellent for **semantic search** and **RAG (Retrieval-Augmented Generation)**.
- It almost directly allows json



Limitations -

- Slightly less contextual nuance on descriptive sentences.

4. **OpenAI Embedding Models (text-embedding-3-small / large):**

Main goal - Provide **universal embeddings** that work across domains — text, code, JSON, documents — for semantic search, clustering, and recommendation.

Trained with Massive multi-domain corpus: web text, documentation, code, Wikipedia, and instruction data. Models learn not only meaning but also *relations* and *metadata semantics*.

Best for JSON because it understands field values well

Strengths

- State-of-the-art semantic accuracy.
- Handles long context windows.
- Excellent for production-scale similarity and clustering.
- Does not directly support json embedding(but process is easy)

Limitations -

- Cloud only (requires API).
- Cost per token (though small models are very cheap).
- No local interpretability or open weights.

5. **Cohere Embed v3:**

Main goal - Provide API-accessible embeddings optimized for document–query similarity and topic clustering.

Strengths -

- Tuned for English semantic similarity.
- Works very well for “find similar items” tasks.
- Provides metadata filtering + re-ranking features in Cohere API.
- Directly allows json embedding

Limitations -

- Cloud API only.

Preferred embedding choice:

Part B:

The preferred embedding choice for us is **BGE Family (BAAI General Embedding)** since it allows us to use json embedding directly(almost) and based on some online info it's best for ecommerce. Since we will be building a chatbot for ecommerce website, we think this embedding will work best for us.

We also intend to take a look at the below embedding in future (to identify the ones which suit us best):

1. **Cohere Embed v3**
2. **E5 Family**

Also we are trying out various embedding logics like, embedding important columns separately and finally concatenating all embeddings we got together, usage of CLIP, etc. We are also **experimenting on the format of data we are going to embed**. Approaches like coming up with a text from features, using JSON/XML will be tested.

Part A:

We are going to use the BGE Family embedder to embed policy docs, since the model also performs well on chunks created from paragraphs.

Re-embedding:

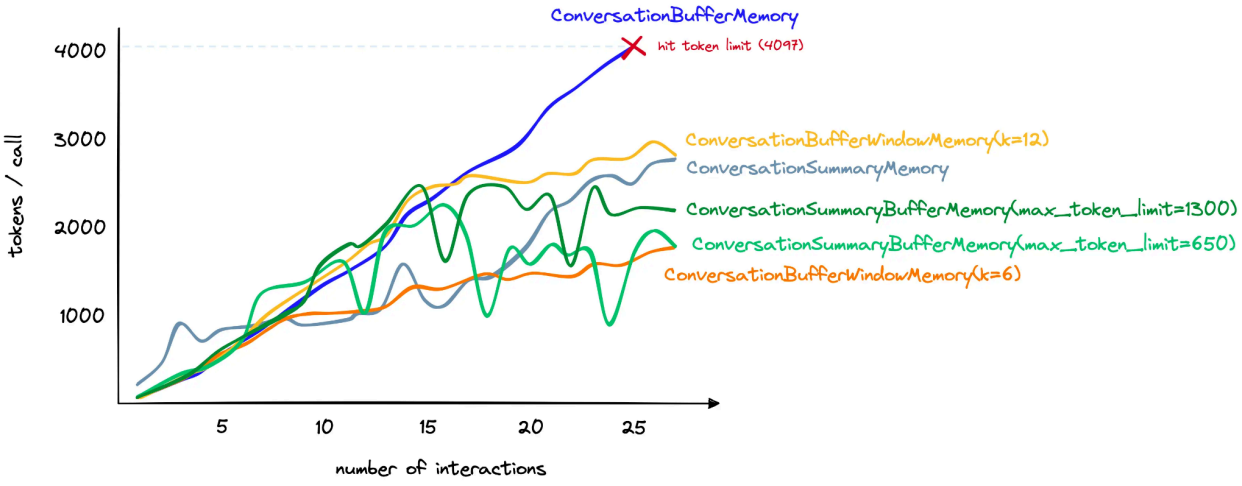
- When changes are made in the embedding doc we will simply delete all the embeddings generated for the doc from the vector database (delete the collection) and re-embed the doc again.
- With respect to the products, if a certain product is removed from the catalog, we will just delete the corresponding embedding.
- If any product's info is edited, we delete the embedding of that product and re-embed it.

How to maintain chat history and feed to LLM?

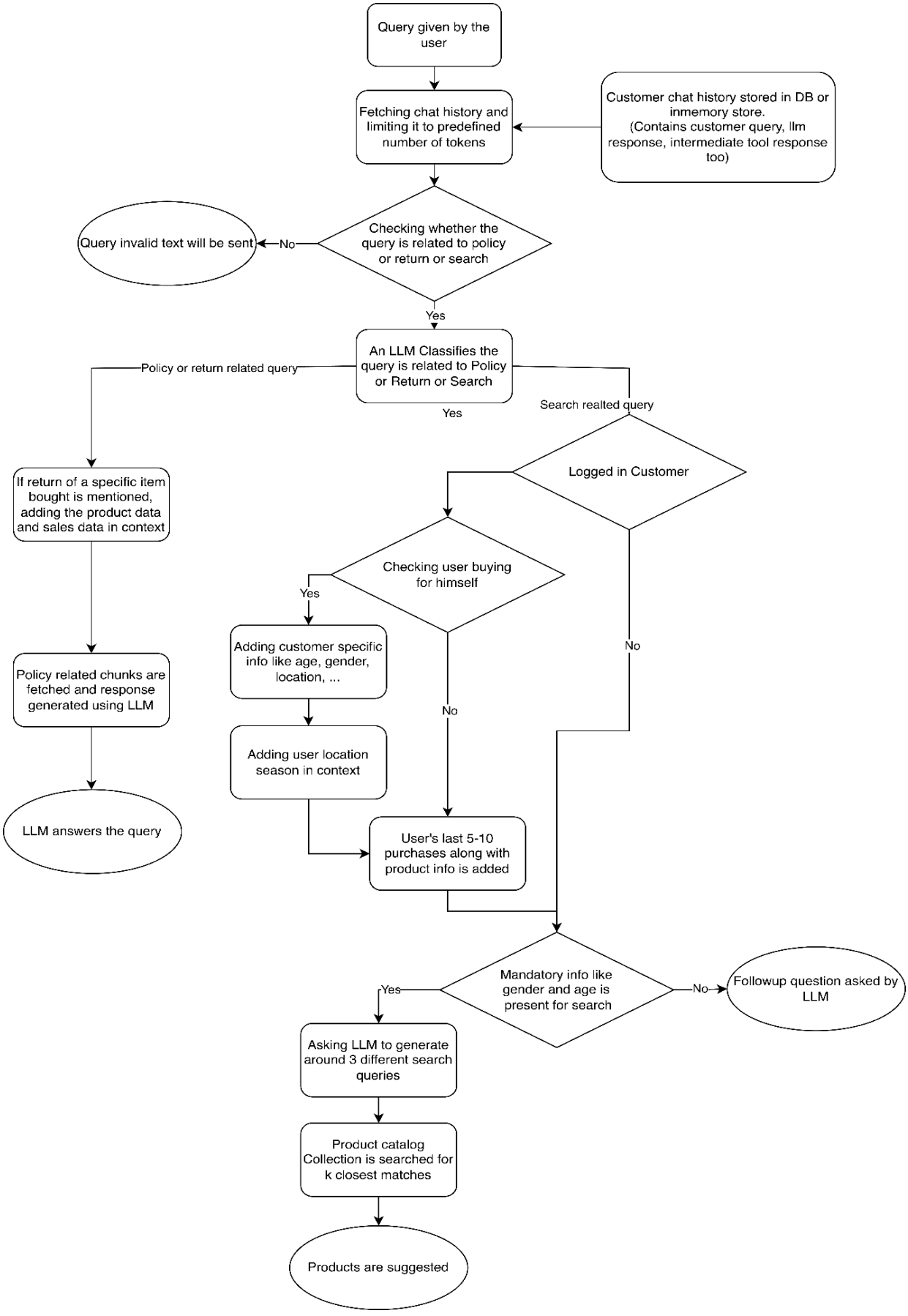
Conversational history : For implementing this task in our chatbot, a conversational memory will be maintained, enabling a coherent conversation.

Breakdown of the pipeline-

- Initialize the *ConversationChain* with a LLM model.
- We will be keeping the conversational memory in one of the parameters for the model in *{history}*, and
- For saving each customer's chat history from a session to a persistence storage, we will be using Redis (planning to save latest session's chat history)
- Since maintaining long chat histories of customers could lead to exceeding the maximum token limit, we will be implementing conversation summarizing techniques.
  The difference between such techniques *(e.g. ConversationSummaryMemory)* and without it *(ConversationBufferMemory)* is shown in the chart below.



Query Flow:


Query given by the user
Fetching chat history and limiting it to predefined number of tokens
Customer chat history stored in DB or inmemory store. (Contains customer query, llm response, intermediate tool response too)
Checking whether the query is related to policy or return or search
No
Query invalid text will be sent
Yes
An LLM Classifies the query is related to Policy or Return or Search
Policy or return related query
Yes
Search realted query
If return of a specific item bought is mentioned, adding the product data and sales data in context
Logged in Customer
Checking user buying for himself
Yes
No
Adding customer specific info like age, gender, location, ...
Adding user location season in context
No
Policy related chunks are fetched and response generated using LLM
LLM answers the query
User's last 5-10 purchases along with product info is added
Mandatory info like gender and age is present for search
Yes
No
Followup question asked by LLM
Asking LLM to generate around 3 different search queries
Product catalog Collection is searched for k closest matches
Products are suggested

Flow Explanation:

Checking whether the query is related to policy or return or search:

LLM will be fed the user query along with history. It will also be fed an instruction that it is a chatbot which will answer about policies of an online clothing store or it will help customers search for clothes. If the query along with the history is related to the tasks provided to the LLM, we will move further else the LLM will ask the customer to ask relevant questions.

An LLM Classifies the query is related to Policy or Return or Search:

- Here LLM is instructed to classify the query into one of the three tasks. The query asks for any info about store policy or any return instruction or it is a search query for clothes. Based on the classification provided by the LLM we will choose a path and move further.
- If the query is a simple policy related query, we will add context via embeddings retriever.
- If the query is related to returning a product, then the product is searched in the product catalog and also the sales data is added to the context along with the context we will be getting from searching the policy doc via embeddings retriever.
- For product search related queries lots of other context needs to be added which will be explained in the following sections.

Context building for product search:

- If the customer is a logged in user, then we can add his/her data like age and gender in context if he is buying for themself.
- If the customer is a logged in user and not buying for themself, we will try to get age and gender from the query, if not possible, the LLM will ask a follow-up question. The same approach will be used in case of guest users.
- If we are able to get location data (can get it from user DB), an api call to fetch the season of a particular zipcode can be used. This season info can be added into context.
- For logged in customers we also have previous sales data. Using that data we can add into context, at what price point the customer purchases products, how much discount was there on the products, category, sub category and style of products purchased, etc.

- If from the context built if all the mandatory info is present, we will ask LLM to come up with 2-3 search queries using the context and search for relevant products in Vector DB. If mandatory info is not available in context then a followup question will be asked by the LLM.

Re-ranking:

The vector database will return multiple products for a search query. We will rerank that using the product info and context built. Currently we think we can use an LLM to do it. But we need further analysis in this part and will decide the approach in future.